

SAPIENZA
UNIVERSITÀ DI ROMA

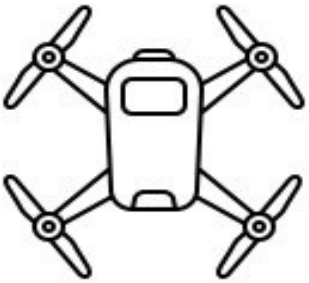
Software Engineering

AY 2023/2024

Salauchuk Maksim salauchuk.1764024@studenti.uniroma1.it

Computer Science Department, Sapienza university of Rome

Controllo formazione droni



1. Introduzione

Obiettivi del Progetto:

- Progettare un sistema di controllo per una formazione di droni destinati alla sorveglianza continua di un'area di 6×6 km.
- Garantire che ogni punto dell'area sia verificato da almeno un drone ogni 5 minuti.
- Gestire l'autonomia limitata dei droni (30 minuti di volo) e i tempi di ricarica variabili (tra 2 e 3 ore).
- Sviluppare un sistema modulare e scalabile, con processi comunicanti e registrazione dei log per la validazione e il monitoraggio.

Struttura della Relazione:

- La relazione è organizzata in sezioni che descrivono il sistema, i requisiti, l'implementazione e i risultati ottenuti.
- Include diagrammi UML per illustrare il design del sistema.
- Fornisce dettagli sul software sviluppato e sui test effettuati.

2. Descrizione generale

2.1 Descrizione del Sistema

Panoramica del Sistema:

Il sistema è composto da un centro di controllo che coordina una flotta di droni per la sorveglianza di un'area specifica.
I droni pattugliano l'area seguendo rotte pianificate per assicurare una copertura completa entro il tempo stabilito.

Il centro di controllo gestisce la comunicazione con i droni, monitora il loro stato e assegna le missioni in base alle esigenze operative.

Funzionalità Principali:

- **Gestione della Copertura**: Assicurare che ogni punto dell'area sia controllato ogni 5 minuti.
- **Monitoraggio dei Droni**: Tracciare in tempo reale la posizione, lo stato di carica e le attività dei droni.
- **Gestione delle Ricariche**: Coordinare il rientro dei droni per la ricarica e l'assegnazione di droni sostitutivi.
- **Comunicazione**: Utilizzare Redis per lo scambio rapido di informazioni tra il centro di controllo e i droni.
- **Registrazione dei Log**: Salvare i dati real-time su Redis (dump.rdb) e registrare le metriche di performance della simulazione in file CSV

Contesto Operativo:

- I droni operano in un'area di 6×6 km con il centro di controllo situato al centro.
- Ogni drone ha un'autonomia di volo di 30 minuti e una velocità di 30 km/h.
- I tempi di ricarica dei droni variano casualmente tra 2 e 3 ore per simulare condizioni reali.

2.2 Architettura del sistema

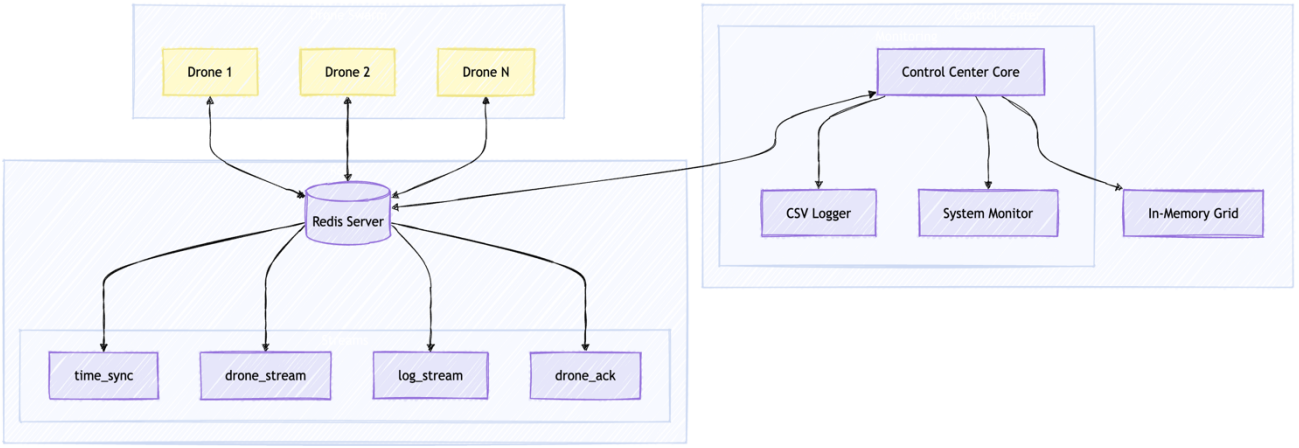


Figura 1.1: Architettura del Sistema di Monitoraggio dei Droni

Questa figura illustra l'architettura complessiva del sistema di monitoraggio dei droni:

Control Center (CC): Il cuore del sistema di controllo che interagisce con i dati e i flussi di comunicazione.

Redis (RS): Il server Redis, che gestisce i flussi di comunicazione attraverso quattro stream distinti: *time_sync*, *drone_stream*, *log_stream*, e *drone_ack*.

Drone Fleet: Una flotta di droni (Drone 1, Drone 2, e Drone N) che inviano e ricevono dati dal server Redis, permettendo una gestione centralizzata delle operazioni.

3.1 User requirements

Requisiti funzionali:

- **Sorveglianza continua dell'area**

  Il sistema deve garantire una sorveglianza continua e affidabile di un'area designata, verificando ogni punto almeno una volta entro un intervallo di tempo specifico.

- **Gestione automatica delle rotte dei droni**

  Il sistema deve assegnare automaticamente le rotte ai droni per ottimizzare la copertura dell'area, senza richiedere interventi manuali.

- **Gestione autonoma delle batterie**

  Il sistema deve gestire automaticamente i livelli di batteria dei droni, garantendo che non si interrompa la sorveglianza e che non sia necessario l'intervento umano per la ricarica.

- **Affidabilità nella comunicazione**

  Il sistema deve garantire una comunicazione in tempo reale tra i droni e il Control Center per coordinare i movimenti e la raccolta dati.

Use Case 1: Monitoraggio Continuo dell'Area

Il sistema consente di monitorare continuamente un'area designata, assicurando che ogni punto venga verificato almeno una volta entro un intervallo di tempo predefinito.

Attori principali:

Control Center: Assegna tratte ai droni e monitora i loro progressi.

Drone: Esegue la tratta assegnata e invia i dati di monitoraggio al Control Center

Pre-condizioni:

I droni devono essere attivi e connessi al sistema.

Il Control Center deve conoscere la mappa dell'area da sorvegliare.

Flusso principale:

Il Control Center divide l'area in tratte e assegna le rotte ai droni disponibili.

I droni eseguono la sorveglianza inviando dati al Control Center.

Il Control Center verifica che tutte le tratte siano state completate.

Obiettivi dell'utente:

Assicurarsi che l'area designata sia costantemente monitorata senza lacune nella copertura.

Ricevere aggiornamenti regolari sullo stato della sorveglianza.

Use Case 2: Gestione Automatica della Batteria

Il sistema monitora i livelli di batteria dei droni e gestisce automaticamente i cicli di ricarica, senza richiedere interventi manuali da parte dell'utente.

Attori principali:

Control Center: Monitora lo stato delle batterie dei droni.

Drone: Invia aggiornamenti sul livello di batteria e si dirige alla base quando necessario.

Pre-condizioni:

I droni devono inviare periodicamente i dati sul livello della batteria.

Devono essere disponibili stazioni di ricarica al centro dell'area.

Flusso principale:

Il drone monitora continuamente il proprio livello di batteria.

Quando il livello scende sotto una soglia definita, il drone invia una notifica al Control Center.

Il Control Center pianifica la ricarica del drone senza compromettere la sorveglianza dell'area.

Obiettivi dell'utente:

Garantire la continuità del monitoraggio anche durante i cicli di ricarica.

Assicurarsi che i droni non rimangano mai senza batteria.

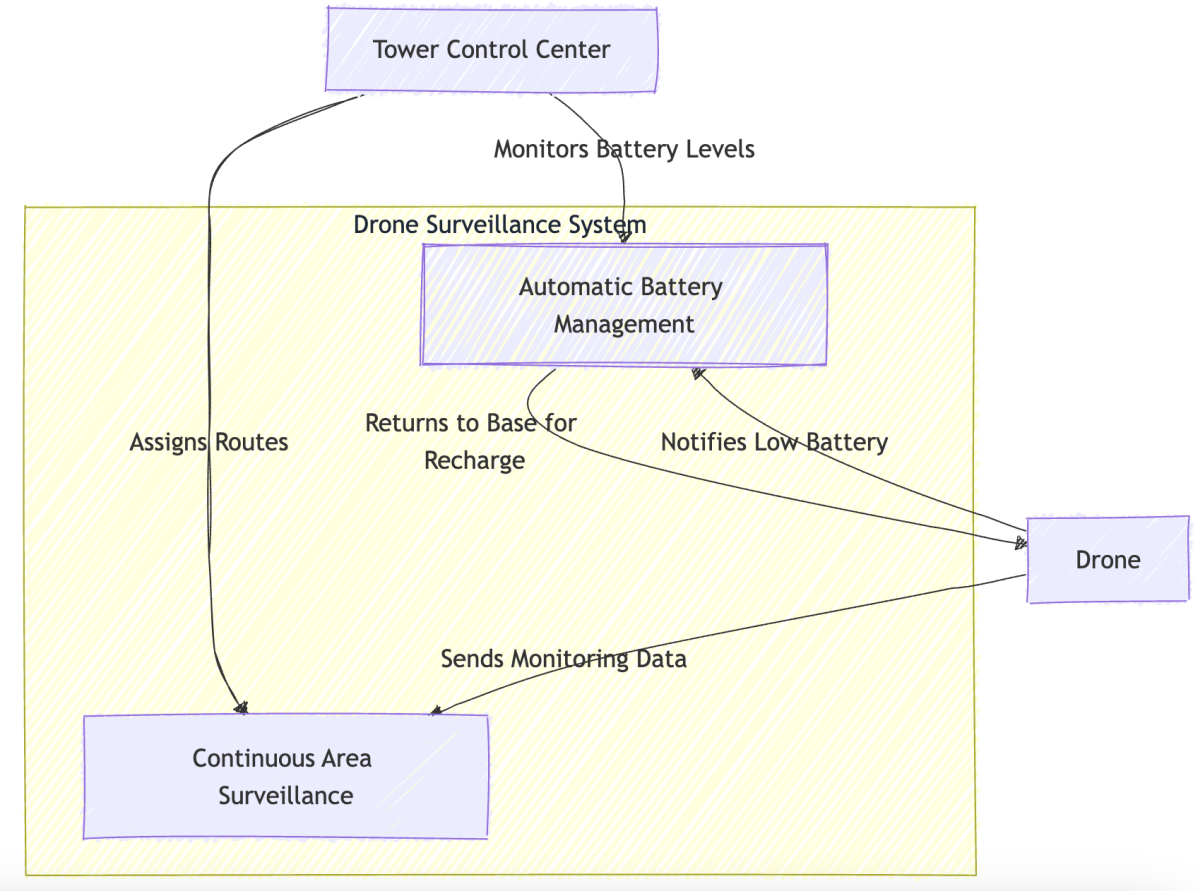


Figura 3.1 - Diagramma UML per il sistema di sorveglianza dei droni. Questo diagramma rappresenta le interazioni tra il Control Center e i droni, con i due casi d'uso principali: *Continuous Area Surveillance* e *Automatic Battery Management*. I droni comunicano con il centro per l'assegnazione delle rotte e per la gestione automatica della batteria.

3.2 System requirements

L'obiettivo principale del sistema è garantire, tramite il Control Center, il monitoraggio di un'area di 6x6 km. L'area viene suddivisa logicamente in sottoaree, gestite e monitorate attraverso il coordinamento e il controllo di droni.

Requisiti funzionali:

- Il sistema deve gestire la sorveglianza di un'area 6000m x 6000m
- Il sistema deve garantire la verifica di ogni punto della griglia almeno una volta ogni 5 minuti
- L'area deve raggiungere il 99% di copertura entro 15 minuti dall'avvio
- Il sistema deve monitorare continuamente i livelli di batteria dei droni, pianificando in anticipo la ricarica per garantire che non ci siano interruzioni nella copertura e garantendo e che il livello della batteria non scenda mai al di sotto del margine minimo di sicurezza pari a 1.0%.
- Il sistema deve mantenere la copertura durante le sostituzioni dei droni.
- Il sistema deve utilizzare stream Redis per la comunicazione tra componenti.
- Ogni drone deve avere uno stream di comunicazione dedicato.

Requisiti non funzionali:

- **Prestazioni**

  Il tempo di latenza nella comunicazione tra droni e centro di controllo deve essere inferiore a 100ms

  Il sistema deve operare continuamente per almeno 35,540 time step (≈ 9.87 ore)

- **Capacità e scalabilità**

  Il sistema deve supportare fino a 6.000 droni simultaneamente

- **Autonomia dei droni**

  Ogni drone deve avere una distanza massima di volo di 15,000 metri con batteria piena

  I droni devono completare la ricarica in un tempo compreso tra 2 e 3 ore

- **Affidabilità della copertura**

  Il sistema deve garantire una copertura del 99% dell'area in ogni momento

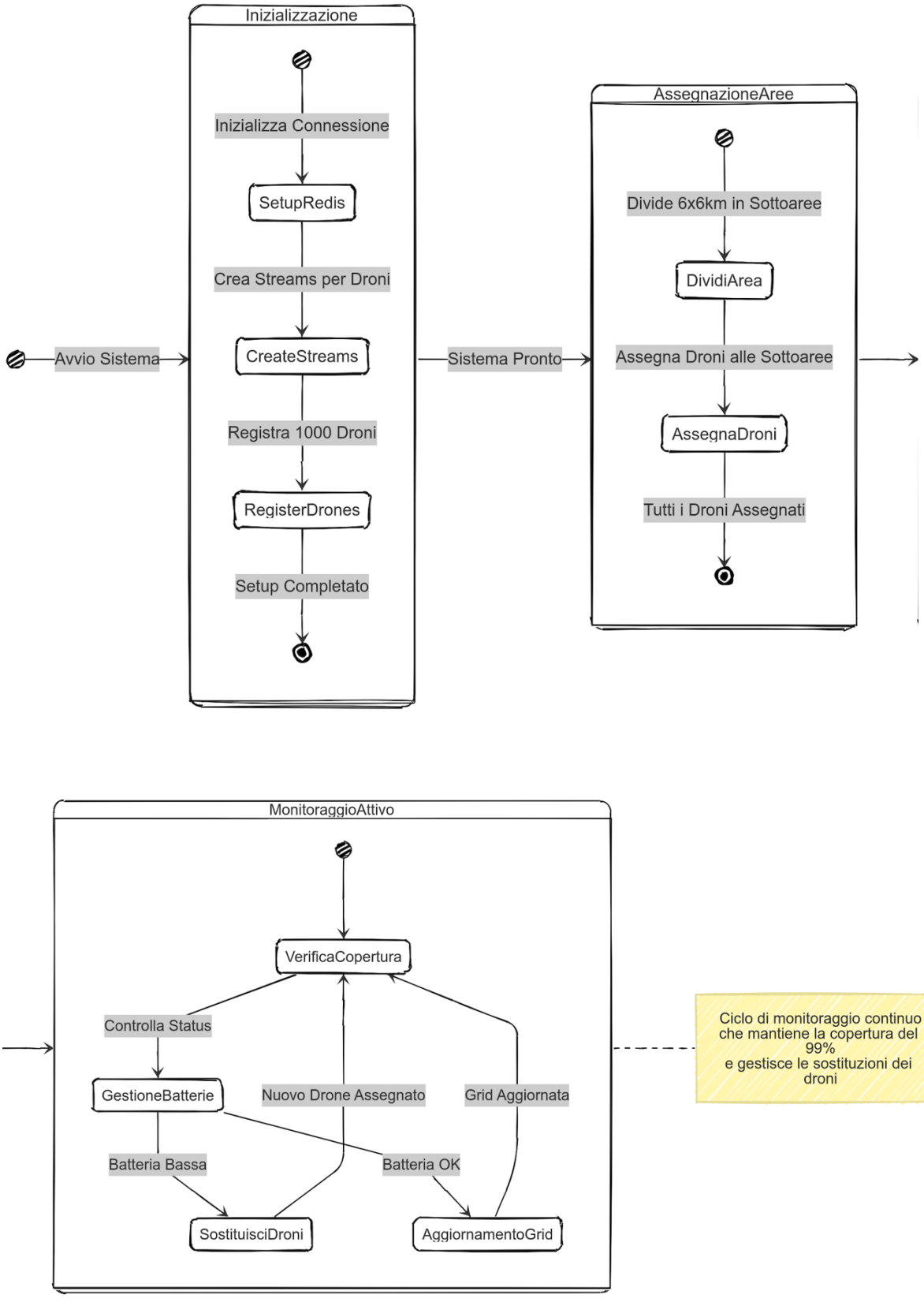


Figura 3.2 – Activity Diagram

Mostra il flusso complessivo del sistema, dall'inizializzazione al monitoraggio continuo dell'area, evidenziando come le componenti collaborano per mantenere la copertura richiesta. Le principali attività includono l'assegnazione delle rotte ai droni, il controllo dello stato della batteria e il monitoraggio costante dell'area da sorvegliare.

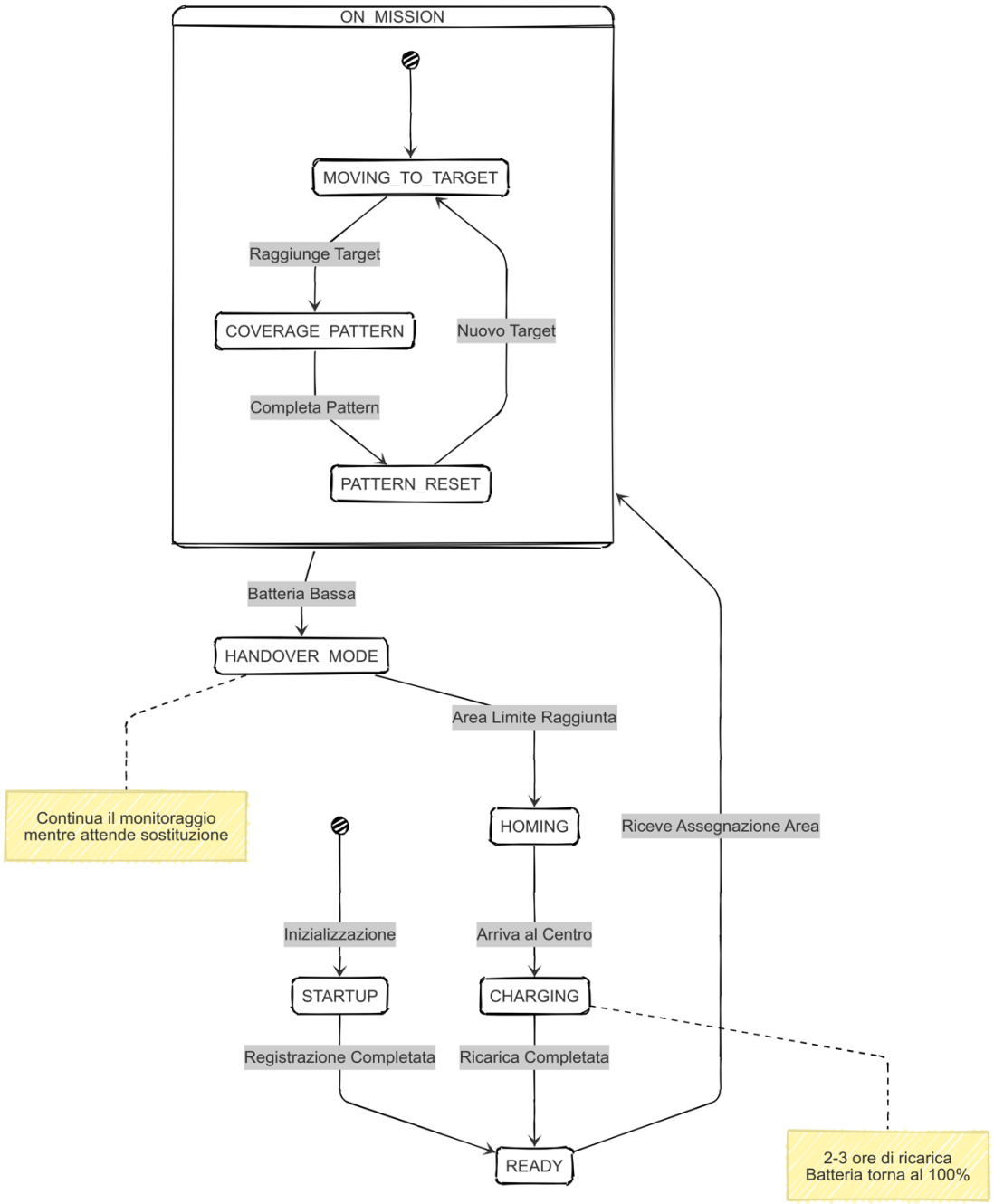


Figura 3.3 – State Diagram

Rappresenta gli stati possibili di un drone e le transizioni tra questi stati, inclusi i meccanismi di gestione della batteria e il pattern di copertura dell'area. Gli stati principali includono *In volo*, *In ricarica*, e *Attesa di istruzioni*, con transizioni condizionate dallo stato della batteria e dalla copertura richiesta.

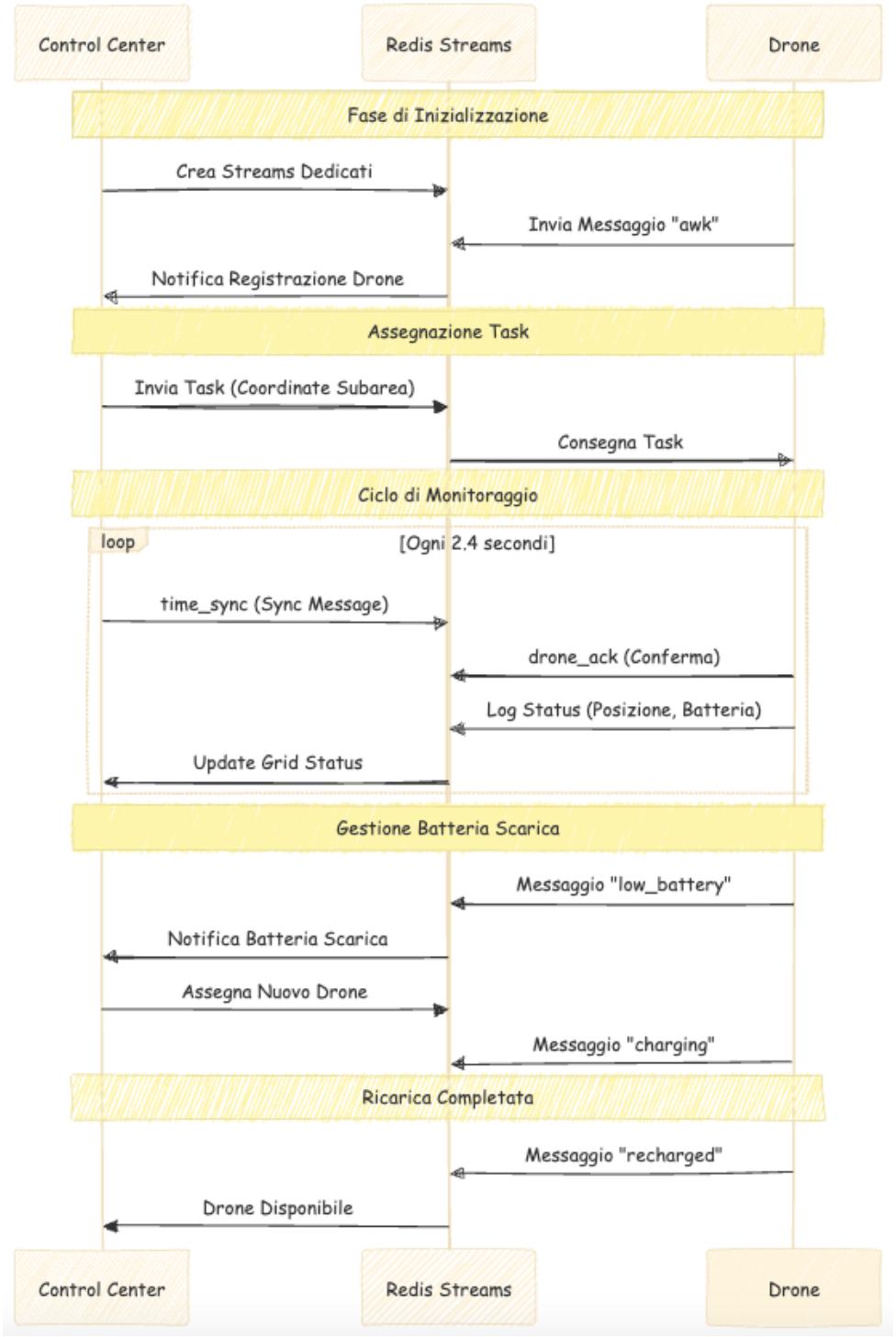


Figura 3.4 – Message Sequence Chart

Illustra il protocollo di comunicazione tra il Control Center e i droni attraverso Redis Streams, mostrando i messaggi scambiati durante le varie fasi operative. Include richieste di aggiornamento dello stato, invio delle rotte di sorveglianza e notifiche sul completamento delle attività o sulla necessità di ricarica.

4. Implementation

4.1 Descrizione Generale

Il sistema è implementato in C++ e utilizza Redis come middleware per la comunicazione tra il Centro di Controllo e i Droni. La struttura principale è divisa in due componenti principali:

1. **Control Center (CC)**: Gestisce la flotta di droni e coordina le operazioni di monitoraggio
2. **Drone**: Implementa la logica di volo e monitoraggio del singolo drone

4.2 Pseudo-codice Componenti

Control Center

```
class ControlCenter {
    grid[300][300]     // Griglia che tiene traccia delle verifiche
    drones[]           // Array dei droni gestiti
    job_queue[]        // Coda dei job in attesa di assegnazione

    init() {
        inizializza_connessione_redis()
        inizializza_grid()
        configura_streams_redis()
    }

    simultation_step() {
        switch(status) {
            case STARTUP:
                registra_droni()
                if (droni_registrati == DRONES_COUNT)
                    status = READY
            case READY:
                assegna_subaree_ai_droni()
                status = RUNNING
            case RUNNING:
                gestisci_messaggi_droni()
                gestisci_sostituzioni_batteria()
        }
    }

    log(timestep) {
        per ogni drone:
            leggi_stato()
            verifica_batteria()
            aggiorna_grid()
        verifica_area_coperta()
```

```
        salva_statistiche()
    }
}

```

Drone

```
class Drone {
    position(x,y)    // Posizione corrente
    battery          // Livello batteria
    status          // Stato operativo
    job             // Task corrente

    simulation_step() {
        switch(status) {
            case STARTUP:
                registra_con_cc()
            case READY:
                attendi_job()
            case ON_MISSION:
                if (batteria_bassa())
                    richiedi_sostituzione()
                esegui_movimento()
            case HANDOVER_MODE:
                continua_fino_limite()
            case HOMING:
                ritorna_base()
            case CHARGING:
                ricarica()
        }
    }

    move() {
        calcola_velocità()
        aggiorna_posizione()
        verifica_completamento_pattern()
    }
}

```

Schema Redis

Il sistema utilizza diversi stream Redis per la comunicazione:

```
- drone_ack      // Sincronizzazione droni -> CC
- time_sync      // Sincronizzazione CC -> droni
- drone_stream      // Messaggi di stato droni -> CC
- log_stream        // Log dei droni per monitoraggio
- [0..999]         // Stream individuali per ogni drone
```

Connessioni Redis

La comunicazione con Redis è gestita utilizzando la libreria hiredis:

```
// Inizializzazione connessione
redisContext *c = redisConnect("127.0.0.1", 6379);

// Configurazione streams
XGROUP CREATE stream_name group_name $ MKSTREAM
```

```
6.
7. // Scrittura messaggi
8. XADD stream_name * field value
9.
10. // Lettura messaggi
11. XREADGROUP GROUP group consumer COUNT 1 STREAMS stream_name >
12.
13. // Pattern di comunicazione:
14. 1. Sincronizzazione temporale
15.    - CC invia sync su time_sync
16.    - Droni rispondono su drone_ack
17.
18. 2. Assegnazione task
19.    - CC scrive su stream individuale del drone
20.    - Drone legge il proprio stream
21.
22. 3. Monitoraggio
23.    - Droni scrivono su drone_stream e log_stream
24.    - CC legge e processa i messaggi
25.
```

Inizializzazione CC:

```
1. // In cc::init_redis()
2. // Crea gli stream base
3. DEL drone_ack_stream
4. stream_initialize_group(drone_ack_stream)  // Crea gruppo "diameter"
5.
6. DEL time_sync
7. stream_initialize_group(time_sync)         // Crea gruppo "diameter"
8.
9. DEL drone_stream
10. stream_initialize_group(drone_stream)      // Crea gruppo "diameter"
11.
12. DEL log_stream
13. stream_initialize_group(log_stream)        // Crea gruppo "diameter"
14.
15. // Crea stream individuali per ogni drone
16. for (i = 0 to DRONES_COUNT) {
17.     DEL i
18.     XGROUP CREATE i diameter $ MKSTREAM    // Stream personale per ogni drone
19. }
20.
```

Fase di Registrazione Droni:

```
1. // Ogni drone in STARTUP:
2. XADD drone_stream * type awk did [id]      // Drone si registra
3.
4. // CC in STARTUP legge le registrazioni:
5. XREADGROUP GROUP diameter cc ... drone_stream  // Legge registrazioni e crea droni
6.
```

Ciclo Principale:

```
1. // 1. Sincronizzazione
2. CC: XADD time_sync * type Sync             // CC invia sync
3. Droni: XREADGROUP GROUP diameter drone ... time_sync  // Droni leggono sync
```

```
Droni: XADD drone_ack * type sync          // Droni confermano
CC: XREADGROUP GROUP diameter cc ... drone_ack  // CC attende conferme

// 2. Assegnazione Task
CC: XADD [drone_id] * type task ...        // CC assegna task ai droni
Droni: XREADGROUP GROUP diameter drones [id] ...  // Ogni drone legge il suo task

// 3. Aggiornamenti Stato
Droni: XADD drone_stream * type low_battery ...  // Droni segnalano batteria scarica
Droni: XADD log_stream * did battery_lvl ...     // Droni loggano stato

CC: XREADGROUP GROUP diameter cc ... drone_stream // CC legge aggiornamenti
CC: XREADGROUP GROUP diameter cc ... log_stream   // CC legge log

```

Shutdown:

```
// Pulizia degli stream
DEL [tutti gli stream dei droni]
DEL drone_stream
DEL drone_ack_stream

```

Raccolta Dati (Test Data Collection):

```
// In cc.cpp
void cc::log_event(int time) {
    // ...
    bool area_verified1 = area_verified;
    area_verified = verify_area(time);
    save_to_csv(calculate_verified_percentage(time), time,TEST_PATH);
    // ...
}

void save_to_csv(double va, int t, std::string filename) {
    std::ofstream file;
    file.open(filename, std::ios::out | std::ios::app);
    file << va << "," << t << "\n";
}

```

Analisi Dati (Test Analysis):

```
// Nel file Python plotter.py
def plot_coverage_analysis(filename, output_filename="system.png"):
    # Read and process the data
    df = pd.read_csv(filename)
    df['minutes'] = df['t'] / 60 // # Converte il tempo da secondi a minuti per leggibilità

```

Visualizzazione (Test Reporting):

```
# Nel file Python plotter.py
    plt.plot(df['minutes'], df['verified_area'], // # Visualizza trend copertura nel tempo
             linewidth=2,
             color='#2E86C1',
             label='Area Coverage')

```

```
    # Verifica requisiti: # - 99% copertura (linea orizzontale)
    plt.axhline(y=99, color='#27AE60', linestyle='--',
                label='Required Coverage (99%)')
      # - Target 15 minuti (linea verticale)
    plt.axvline(x=15, color='#E74C3C', linestyle='--',
                label='Time Target (15 min)')

```

Questa implementazione permette:

- Sincronizzazione temporale tra componenti
- Comunicazione asincrona affidabile
- Gestione efficiente delle sostituzioni per batteria scarica
- Monitoraggio in tempo reale dello stato del sistema

Monitor di Copertura Temporale

- Scopo: Garantire che ogni punto dell'area sia verificato almeno una volta ogni 5 minuti
- Implementazione: Utilizza una griglia dove ogni cella memorizza il timestamp dell'ultima verifica

```
bool is_verified(int last_v, int current_time)
{
    return last_v != -1 && ((double)(current_time - last_v) / T) <= (300.0 / T);
}

```

```
double cc::calculate_verified_percentage(int curr_time) {
    int aa = 0;
    for (int i = 0; i < HEIGHT; i++) {
        for (int j = 0; j < WIDTH; j++) {
            if (!verified_single_point(grid[i][j], curr_time)) {
                continue;  // Punto non conteggiato come verificato
            }
            aa++;  // Incrementa solo punti verificati validi
        }
    }
    return ((double)aa / (300.0 * 300.0)) * 100.0;  // Calcola percentuale
}

```

Monitor di Tempo Massimo di Verifica

- Scopo: Assicurare che l'intera area venga verificata entro 15 minuti dall'avvio
- Implementazione: Verifica il timestamp quando l'area diventa completamente coperta

```
1. if (!area_verified1 && area_verified)
2. {
3.     assert(MAX_VERIFY_TIME >= time*T);  // 900 secondi = 15 minuti
4. }
5.
```

Monitor di Sicurezza Batteria

- Scopo: Controllare che nessun drone operi con batteria sotto la soglia critica dell'1%
- Implementazione: Verifica il livello di batteria di ogni drone attivo

```
1. assert(drones[did].battery > MIN_BATTERY);  // MIN_BATTERY = 1.0
2.
```

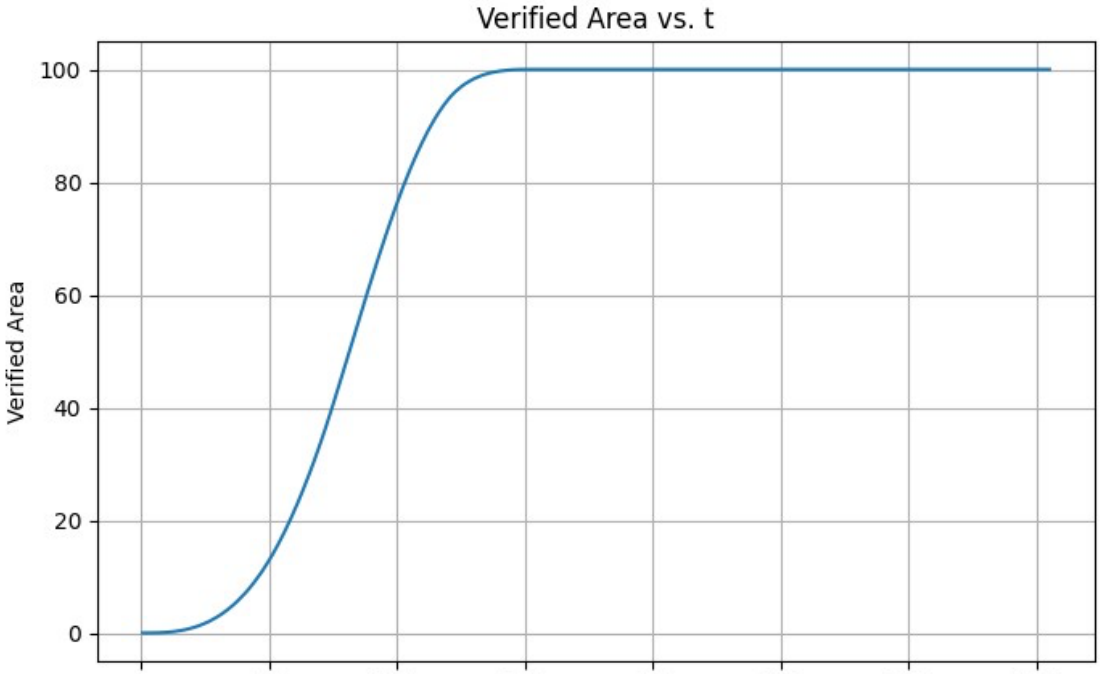
Questi monitor sono integrati nella funzione di logging del sistema che viene eseguita ad ogni timestep:

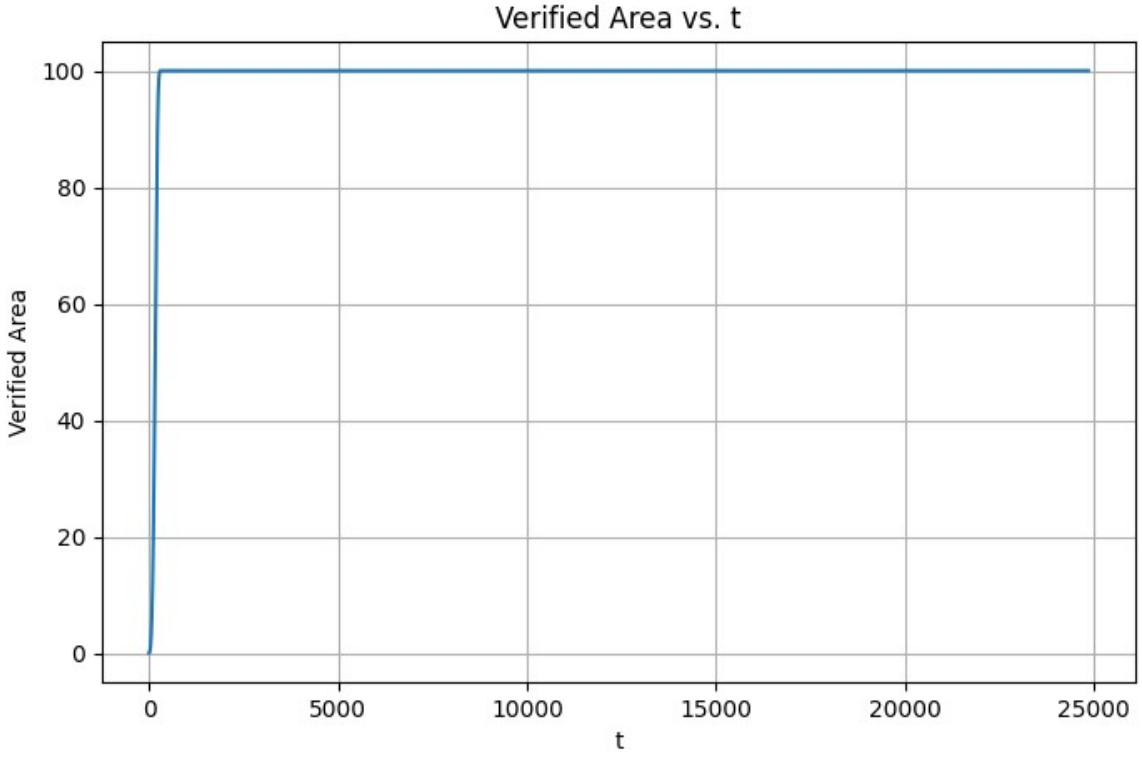
```
 1. void ControlCenter::log(int timeStep)
 2. {
 3.     // Raccolta dati dai droni
 4.     for (int i = 0; i < DRONES_COUNT; i++) {
 5.         // ... lettura stato droni ...
 6.
 7.         // Monitor batteria
 8.         assert(drones[did].battery > MIN_BATTERY);
 9.
10.         // Aggiornamento griglia di copertura
11.         if (grid[ilvy][ilvx] < ltv) {
12.             grid[ilvy][ilvx] = ltv;
13.         }
14.     }
15.
16.     // Monitor area e tempo
17.     bool area_verified1 = area_verified;
18.     area_verified = check_area(time);
19.     if (!area_verified1 && area_verified) {
20.         assert(MAX_VERIFY_TIME >= time*T);
21.     }
22.
23.     // Salvataggio dati
24.     save_to_csv(get_percentage(time), time, TEST_PATH);
25. }
26.
```

Se una qualsiasi di queste condizioni viene violata, il sistema genera un errore di asserzione e interrompe l'esecuzione, segnalando quale requisito critico è stato violato. Questo permette di identificare immediatamente problemi nel funzionamento del sistema che potrebbero compromettere la missione di sorveglianza.

5. Risultati sperimentali

Attraverso l'analisi dei dati, intendiamo individuare tendenze, identificare picchi di carico e valutare l'impatto delle configrazioni del sistema sulle prestazioni complessive.





Sono state eseguite due simulazioni, rispettivamente una da 720 s e una da 25000 s, per poter ottenere i campioni utili al fine della valutazione. Dai test eseguiti si evincono le seguenti conclusioni:

Modello Probabilistico

La probabilità di mantenere una copertura completa può essere modellata come:

$$P(C)=P(D)\times P(T|D)\times P(B|T,D)$$

Dove:

1.**P(D):** Probabilità di avere droni sufficienti per la copertura base:

$$P(D)=\min\left(\frac{N}{N_{required}},1\right)$$

$$N = \text{numero di droni attivi, } N_{required}=900.$$

2.**P(T|D):** Probabilità di rispettare il vincolo temporale:

$$P(T|D)=e^{-\lambda t}$$

$$\lambda=\frac{1}{MTBF}\ (\text{MeanTimeBetweenFailures}),\ t=300 \text{ secondi.}$$

3.**P(B|T,D):** Probabilità di avere droni con batteria sufficiente:

$$P(B|T,D)=\frac{R\times A}{N}$$

Dove R è il fattore di rotazione (R = 6) (Un drone può coprire un'area per circa 30 minuti 30 minuti / 5 minuti = 6 rotazioni). A è il numero di droni disponibili per la sostituzione e N è il numero totale di droni.

$$P(C) = 1 \times 0.92 \times 1 = 0.92\ (92\%)$$

Affidabilità Richiesta:

$$P(C) \geq 0{,}99 \quad (99\ \%\ \text{affidabilità})$$

Dimensionamento della Flotta:

$$N \approx 900 \times 6 \times \frac{1}{0{,}99} \approx 6.000$$

- Per garantire la copertura totale dell'area rispettando il vincolo che un qualunque punto della mappa deve essere scansionato almeno ogni 5 minuti sono necessarie circa 6.000 unità di Droni.

- dall'avvio della simulazione sono necessari al più 15 minuti per garantire che l'intera Area sia sorvegliata.

Qualora i droni non siano sufficienti a mantenere l'area verificata la simulazione avrà l'esito riportato nella seguente immagine:

